

Modulo 3

Programación en Lenguaje C

- 3.i. Entorno MinGWStudio. Estructura de un programa en C.**
- 3.ii. Entrada/Salida en C.**
- 3.iii. Estructuras de control.**
- 3.iv. Ejercicio de integración.**
- 3.v. Vectores.**
- 3.vi. Matrices.**
- 3.vii. Punteros.**
- 3.viii. Funciones.**
- 3.ix. Ejercicio de integración.**

3.i. Entorno MinGWStudio. Estructura de un programa en C

a) Estructura de un programa en C

```
declaraciones globales
main()
{
    variables locales
    secuencia de sentencias
}
```

```
int main()
{
    char a;
    int b;

    a='a';
    b=3;

    return 0;
}
```

Todos los programas en C son una o más funciones. **La única función que siempre tiene que estar es la función main(),** siendo la primera que es llamada cuando comienza la ejecución del programa. **Los paréntesis encierran los argumentos (puede no haber ninguno) y las llaves indican donde se encuentran las sentencias que componen la función.**

32 palabras clave definidas por el estándar ANSI

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
cgar	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

3.i. Entorno MinGWStudio. Estructura de un programa en C

- **Identificador:** Es un nombre usado para referirse a las variables, las funciones y otros objetos definidos por el usuario.
- **Tipos de datos:** Existen cinco tipos de datos atómicos en C:

<u>Tipo</u>	<u>Tamaño</u>	<u>Rango</u>	
char	8	0 a 255	carácter
int	16	-32768 a 32767	entero
float	32	3.4e-38 a 3.4e+38	real
double	64	1.7e-308 a 1.7e+308	real de doble precisión
void	0	sin valor (uso en funciones principalmente)	

- **Modificadores de tipo:** Los tipos de datos básicos pueden tener distintos modificadores.

<i>signed</i>	con signo	<i>long</i>	largo
<i>unsigned</i>	sin signo	<i>short</i>	corto

3.i. Entorno MinGWStudio. Estructura de un programa en C

<u>TIPO</u>	<u>Bits</u>	<u>Rango</u>
char	8	-128 a 127
unsigned char	8	0 a 255
signed char	8	-128 a 127
int	16	-32768 a 32767
unsigned int	16	0 a 65.535
signed int	16	-32768 a 32767
short int	16	-32768 a 32767
unsigned short int	16	0 a 65.535
signed short int	16	-32768 a 32767
long int	32	-2147483648 a 2147483647
signed long int	32	-2147483648 a 2147483647
float	32	3.4 E -38 a 3.4 E +38
double	64	1.7 E -308 a 1.7 E +308
long double	64	1.7 E -308 a 1.7 E +308

3.i. Entorno MinGWStudio. Estructura de un programa en C

■ Declaración de variables:

tipo_dato lista_variables;

int x,y,z;

int x;

int y;

int z;

short int si;

double balance,beneficio;

unsigned int ui;

static int num;

const pi;

■ Sentencia de asignación:

nombre_variable = expresión;

nombre_variable1 = ... = nombre_variablenn = expresión;

■ Asignación y definición simultáneas:

tipo_dato nombre_variable = expresión;

char car='a';

int primero=0;

float balance=123.23;

3.i. Entorno MinGWStudio. Estructura de un programa en C

- **Constantes:** Valores fijos que no pueden ser alterados por el programa.

Constantes enteras: secuencias de dígitos (0..9).

Constante reales: [dígitos][.dígitos][E/e[-/+]dígitos]

Constantes de carácter: 'a', '%'.

Constantes de carácter de barra invertida: '\n', '\t'.

Constantes de cadena: Secuencias de caracteres encerradas entre dobles comillas.

- **Operadores:**

- Hay operadores *unarios*, *binarios* y *ternarios*.

- Hay de diferentes tipos: *aritméticos*, *relacionales*, *lógicos* y *a nivel de bit*.

- **Operadores aritméticos:**

= Asignación

x=5;

- Resta, también menos monario

a=b-c;

a= -7;

+ Suma

a=b+c;

* Multiplicación

a=b*c;

/ División

a=b/c;

% Resto de una división

a=b % c;

10 % 3 es 1

-- Decremento

a++;

Incrementa a en 1

++ Incremento

b--;

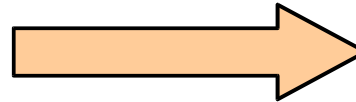
Decrementa b en 1

3.i. Entorno MinGWStudio. Estructura de un programa en C

```
#include<stdio.h>
int main(void)
{
    int i=7,j=49,r;

    r=i+j;
    r=i/j;
    r=i%j;
    r++;
    r--;

    return 0;
}
```



```
r=i+j;  --> 56
r=i/j;  --> 0
r=i%j;  --> 7
r++;    --> 8
r--;    --> 7
```

3.i. Entorno MinGWStudio. Estructura de un programa en C

■ Operadores a nivel de bits:

- Se refieren a operaciones de comprobación, asignación o desplazamiento.
- Operan bit a bit.
- Se aplican sólo a los tipos char e int con sus diferentes variantes.

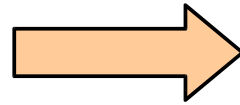
<u>operador</u>	<u>acción</u>		
&	Y	C = A & B	
	O	C = A B	
^	O exclusiva (XOR)	C = A ^ B	
~	Complemento a 1	C = ~A	
>>	Desplazamiento a la derecha	C = A >> 1	(1 posición)
<<	Desplazamiento a la izquierda	C = B << 2	(2 posiciones)

3.i. Entorno MinGWStudio. Estructura de un programa en C

```
#include<stdio.h>
int main(void)
{
    int i=7,j=49,r;

    r=i&j;
    r=i|j;
    r=i>>1;
    r=i<<2;

    return 0;
}
```



i = 00000000 00000111
j = 00000000 00110001

AND= 00000000 00000001 = UNO

OR = 00000000 00110111 = 55

>> = 00000000 00000011 = 3

<< = 00000000 00011100 = 28

3.i. Entorno MinGWStudio. Estructura de un programa en C

■ Otros operadores:

- (y) aumentan la precedencia de las operaciones que contienen.
- Operador de encadenamiento: ,

EJEMPLO: $x = (y = 5, y + 2);$

■ Expresiones:

- Son combinaciones válidas de variables, constantes y operadores.
- Si se mezclan constantes y variables de distintos tipos, todas se convierten a un tipo único que es el que más ocupa.

- **Moldes o casts:** Se utilizan para forzar a que una expresión sea de un tipo determinado.

(tipo_dato) expresión

EJEMPLO: $(\text{float}) x/2;$

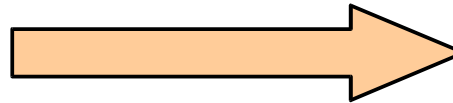
3.i. Entorno MinGWStudio. Estructura de un programa en C

```
#include<stdio.h>
int main(void)
{
    int i=7,j=50;
    float r1,r2;

    r1=j/i;

    r2=(float) j/i;

    return 0;
}
```



```
r=j/i;           --> 7.00
r=(float)j/i;    --> 7.14
```

3.i. Entorno MinGWStudio. Estructura de un programa en C

<i>Precedencia operadores</i>	
Aritméticos	Lógicos y relacionales
* /	!
- +	&
%	
	Relacionales

$R = 4 + 3 * 8$

$(4 + 3) * 8 = 96$

$4 + (3 * 8) = 28$

8.1. Entorno MinGWStudio. Estructura de un programa en C

■ Formateado y documentación de un programa:

- Para dar mayor legibilidad se pueden añadir **tabulaciones y espacios** en una expresión de C.

`x=10/y~(127/x);` `x = 10 / y ~ (127/x);`

- El **uso de paréntesis redundantes** no produce errores ni disminuye la velocidad de ejecución de una expresión.

`x=y/3-34*temp&127;` `x = (y/3) - (34*(temp & 127));`

- El **código contenido entre llaves**, correspondiente a un bloque del programa, **debe sangrarse para mayor legibilidad**.

- **/* */** Inicio y fin de comentario. Los comentarios no se pueden anidar.

`/* Este programa imprime... */`

`/*`

`comentarios ....`

`*/`

3.ii. Entrada/salida en C

- La E/S en C se hace a través de llamadas a funciones de la librería estándar. Esta librería se incluye con todos los compiladores y suele siempre tener las mismas funciones.
- Para poder llamar a estas funciones de E/S hay que incluir en el programa el fichero `STDIO.H`. Esto se hace con la palabra *include*.

`#include <stdio.h>`

- El estándar ANSI C no define funciones que realicen operaciones de control de la pantalla (como la colocación del cursor) o que muestren gráficos, ya que estas operaciones varían mucho dependiendo de las máquinas. Sin embargo, la mayoría de los compiladores incluyen en sus bibliotecas funciones de control de pantalla y de gráficos que se aplican en el entorno específico para el que el compilador se ha diseñado.

3.ii. Entrada/salida en C

ESCRITURA Y LECTURA DE CARACTERES

- **getchar()**: Lee un carácter desde el teclado, habiendo que pulsar <enter> para que la entrada sea realmente enviada al programa. No es adecuada para entrada interactiva.

variable=getchar();

- **putchar()**: Escribe un carácter en el dispositivo de salida estándar.

putchar(variable);

```
#include <stdio.h>
main()
{
    char opcion;
    printf("Dame un carácter \n");
    opcion=getchar();
}
```

```
#include <stdio.h>
#include <conio.h>
main()
{
    char car;

    car=getch();
    putchar(car);
}
```

3.ii. Entrada/salida en C

ESCRITURA Y LECTURA DE CARACTERES

CONSTANTES DE CARÁCTER

\b	retroceso
\f	alimentación de hoja
\n	nueva línea
\r	retorno de carro
\t	tabulador horizontal
\"	doble comilla
\'	simple comilla
\0	nulo
\\	barra invertida
\v	tabulador vertical
\a	alerta
\N	constante octal (N es la constante)
\x	constante hexadecimal

3.ii. Entrada/salida en C

ESCRITURA Y LECTURA DE CADENAS

- Como en C el tipo de dato cadena no existe, éstas se simbolizan mediante vectores de caracteres (los vectores se estudiarán más adelante). En una **cadena de caracteres el último carácter es el carácter nulo '\0'**.
- **gets()**: Lee una cadena de caracteres desde el dispositivo estándar.
gets(variable);
- **puts()**: Escribe un carácter en el dispositivo de salida estándar.
puts(variable);

```
#include <stdio.h>
main()
{
    char linea[80];
    gets(linea);
    puts(linea);
}
```

3.ii. Entrada/salida en C

ESCRITURA Y LECTURA CON FORMATO

- **printf()**: Función de escritura en el dispositivo estándar (pantalla).

printf("cadena de control", lista de argumentos);

EJEMPLO: printf("Hola, me llamo Miguel Indurain\n");

- En la cadena de control aparecen órdenes de formato, que comienzan con un % seguido de un código de formato. Cada orden de formato va ligada sólo a un argumento.

printf("Me gusta %c %s", 'C', "mucho");

Por pantalla saldrá

Me gusta C mucho

3.ii. Entrada/salida en C

ESCRITURA Y LECTURA CON FORMATO

ESPECIFICADORES DE FORMATO DE *printf()*

%c	carácter
%d, %i	enteros decimales con signo
%e, %E	notación científica (e minúscula o E mayúscula)
%f	coma flotante
%g	usar %e o %f, el formato más corto
%G	igual que %g, pero utilizando la E (mayúscula)
%s	cadena de caracteres
%u	enteros decimales sin signo
%x, %X	hexadecimales sin signo (letra minúscula o mayúscula)
%p	mostrar un puntero
%%	imprimir %

Modificadores de formato de *printf*

-	Alinea a la izquierda	%-s
n°	Número mínimo de caracteres a escribir	%5s
.n°	Escribe ese número de decimales si el dato es <i>float</i>	%6.4f
h	Imprime enteros cortos cuando va con el formato i	%hi
l	Enteros largos	%li
L	Escribe datos de tipo <i>long double</i> . Se usa con los códigos <i>e</i> , <i>f</i> , <i>g</i>	%Lf

3.ii. Entrada/salida en C

ESCRITURA Y LECTURA CON FORMATO

```
#include <stdio.h>
main()
{
    int i=2;
    double x=1.5;
    char rep='A';

    printf("[%d]\n",i);
    printf("[%3d]\n",i);
    printf("[% -3d]\n",i);
    printf("[%03d]\n",i);
    printf("[%e]\n",x);
    printf("[%f]\n",x);
    printf("[%6.3f]\n",x);
    printf("[% -6.3f]\n",x);
    printf("[%x]\n",rep);
    printf("[%2x]\n",rep);
    printf("[%02x]\n",rep);
    printf("[%o]\n",rep);
    printf("[%c]\n",rep);
    printf("[%3c]\n",rep);
    printf("[% -3c]\n",rep);
    printf("[%02x]\n",rep);
    printf("COMILLAS \"\n");
    printf("PORCENTAJES %%\n",rep);
```



```
[2]
[  2]
[2  ]
[002]
[1.500000e+00]
[1.500000]
[ 1.500]
[1.500 ]
[41]
[41]
[41]
[101]
[A]
[  A]
[A  ]
[41]
COMILLAS "
PORCENTAJES %
```

3.ii. Entrada/salida en C

ESCRITURA Y LECTURA CON FORMATO

- **scanf()**: Lee datos introducidos por el dispositivo estándar (teclado).

scanf("cadena de control", lista de argumentos);

- En la **cadena de control** aparecen los especificadores de formato.
- Los **argumentos de la lista de argumentos** deben ir precedidos de **&**.
- Si en la cadena de control hay un blanco, se leerán los blancos pero no se almacenarán en la variable leída. Lo mismo sucede con las comas.

scanf("%i",&num);

scanf("%i %f",&numa,&numb);

3.ii. Entrada/salida en C

ESCRITURA Y LECTURA CON FORMATO

ESPECIFICADORES DE FORMATO DE *scanf()*

%c	leer un único carácter
%d, %i	leer un entero decimal
%e, %f, %g	leer un número de tipo coma flotante
%o	leer un número octal
%s	leer una cadena de caracteres
%x	leer un número hexadecimal
%p	leer un puntero
%u	leer un entero sin signo

Modificadores de formato de *scanf*

l	lee un entero largo	%li
h	lee un entero corto	%hi
l	con los especificadores <i>e, f, g</i> lee una variable <i>double</i>	%lg
L	lee una variable <i>long double</i>	%L

3.i. Entorno MinGWStudio. Estructura de un programa en C

■ Operadores relacionales y lógicos:

cierto cualquier valor distinto de 0 (incluidos negativos)
falso el valor 0

Operadores relacionales

<u>operador</u>	<u>acción</u>
>	mayor que
>=	mayor o igual que
<	menor que
<=	menor o igual que
==	igual
!=	no igual (distinto)

Operadores lógicos

<u>operador</u>	<u>acción</u>
&&	Y
	O
!	NO

Precedencia

!	mayor
>, >=, <, <=	
==, !=	
&&	
	menor